Министерство науки и высшего образования Российской Федерации

**Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ Инженерный бизнес и менеджмент
КАФЕДРА ИБМ3 «Промышленная логистика»

ЛАБОРАТОРНАЯ РАБОТА

ПО ДИСЦИПЛИНЕ:
ПАРАДИГМЫ И КОНСТРУКЦИИ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

НА ТЕМУ:
«Объектно-ориентированные возможности языка Python»

Студент ИБМ3-34Б Громадская А.М.

Преподаватель

2026 г.

Условие задачи

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием pip.
2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке Python 3.
3. Все файлы проекта (кроме основного файла main.py) должны располагаться в пакете lab_python_oop.
4. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета lab_python_oop.

Вариант 9

Написать программу, в которой описана иерархия классов: функция от одной переменной (синус, косинус, тангенс). Базовый класс должен иметь методы получения значения функции для данного значения переменной, а также создания экземпляра класса, представляющего собой производную текущего экземпляра. Продемонстрировать работу всех методов классов всех классов.

Текст программы на языке Python в файле base_function.py

```
import math

class MathFunction:
    """
    Базовый класс для математической функции от одной переменной
    """
    def __init__(self, x: float): #конструктор  селф ссылка на объект класса, x передача переменно типа флоат
        # Приватная переменная (инкапсуляция)
        self.__x = x

    # Свойство (property) для получения значения переменной
    @property
    #функция возвращает значение x
    def x(self):
        return self.__x

    @x.setter
    def x(self, value):
        # Ограничение на данные
        if not isinstance(value, (int, float)):
            raise ValueError("Переменная должна быть числом")
        self.__x = value

    def calculate(self):
```

```
        # Базовый метод вычисления. Наследники(син кос тг) его переопределят.
        return self.x

    def get_derivative(self):
        # Метод получения производной.
        return self

    # Строковое представление
    def __str__(self):
        return f"Функция от x={self.x}"
```

Текст программы на языке Python в файле cos_function.py

```
import math
from .base_function import MathFunction


class CosFunction(MathFunction):
    """Класс функции Косинус."""

    def __init__(self, x: float):
        super().__init__(x)

    # Переопределение метода родителя
    def calculate(self):
        return math.cos(self.x)

    # Производная косинуса = минус синус.
    def get_derivative(self):
        from .sin_function import SinFunction
        return SinFunction(self.x)

    def __str__(self):
        return f"Косинус: cos({self.x}) = {self.calculate():.4f}"
```

Текст программы на языке Python в файле sin_function.py

```
import math
from .base_function import MathFunction

# клас синус наследует от класса матери матфунк
class SinFunction(MathFunction):
    """Класс функции Синус"""

    def __init__(self, x: float): #селф ссылка на объкт х типа флоат
        super().__init__(x) # обращение к род классу выполнить конструктор и
передать значение х

    # Переопределение метода родителя
    def calculate(self):
        return math.sin(self.x)

    # Производная синуса = косинус
    def get_derivative(self):
```

```
        from .cos_function import CosFunction
        return CosFunction(self.x)

    def __str__(self):
        return f"Синус: sin({self.x}) = {self.calculate():.4f}"
```

Текст программы на языке Python в файле tan_function.py

```
import math
from .base_function import MathFunction
from .cos_function import CosFunction

class TanFunction(MathFunction):
    """Класс функции Тангенс."""

    def __init__(self, x: float):
        super().__init__(x)

    def calculate(self):
        # ограничение деления на ноль
        if math.cos(self.x) == 0:
            raise ValueError("Тангенс не определен при cos(x) = 0")
        return math.tan(self.x)

    # Производная тангенса = 1 / cos^2(x).
    def get_derivative(self):
        return CosFunction(self.x)

    def __str__(self):
        return f"Тангенс: tg({self.x}) = {self.calculate():.4f}"
```

Текст программы на языке Python в файле main.py

```
from colorama import Fore, Style, init
from lab_python_oop.sin_function import SinFunction
from lab_python_oop.cos_function import CosFunction
from lab_python_oop.tan_function import TanFunction

# Инициализируем colorama (сторонняя библиотека)
init(autoreset=True)

def main():
    # Выбор типа объекта пользователем
    print("\nВыберите функцию для демонстрации:")
    print("1. Синус")
    print("2. Косинус")
    print("3. Тангенс")

    choice = input("Ваш выбор (1-3): ")


    # Создаем объект в зависимости от выбора
    if choice == '1':
```

```
        try:
            x_val = float(input("Введите значение аргумента x (например, 1.57): "))
        except ValueError:
            print(Fore.RED + "Ошибка: введите число!")
            return
        a = SinFunction(x_val)

    elif choice == '2':
        try:
            x_val = float(input("Введите значение аргумента x (например, 1.57): "))
        except ValueError:
            print(Fore.RED + "Ошибка: введите число!")
            return
        a = CosFunction(x_val)

    elif choice == '3':
        try:
            x_val = float(input("Введите значение аргумента x (например, 1.57): "))
        except ValueError:
            print(Fore.RED + "Ошибка: введите число!")
            return
        a = TanFunction(x_val)

    else:
        print(Fore.RED + "Неверный выбор!")
        return
    print(Fore.GREEN + "\nРезультат работы ")


    print(Fore.GREEN + "\nРезультат работы ")


        # Вызов print для экземпляра класса (сработает __str__)
    print(a)

        # Демонстрация метода базового класса
    print(f"Значение функции: {a.calculate():.4f}")

        # Демонстрация производной
    derivative = a.get_derivative()
    print(f"Производная в этой точке (базовый класс): {derivative}")


if __name__ == "__main__":
    main()
```

Вывод

```
Выберите функцию для демонстрации:
1. Синус
2. Косинус
3. Тангенс
Ваш выбор (1-3): 3у
Неверный выбор!
(venv) PS C:\Users\Nastya\OneDrive\Рабочий стол\Лаба2питон>
```

```
(venv) PS C:\Users\Nastya\OneDrive\Рабочий стол\Лаба2питон> & "c:\Users\Nast
e/Рабочий стол/Лаба2питон/main2.py"

Выберите функцию для демонстрации:
1. Синус
2. Косинус
3. Тангенс
Ваш выбор (1-3): 1
Введите значение аргумента x (например, 1.57): 1

Результат работы
Синус: sin(1.0) = 0.8415
Значение функции: 0.8415
Производная в этой точке (базовый класс): Косинус: cos(1.0) = 0.5403
(venv) PS C:\Users\Nastya\OneDrive\Рабочий стол\Лаба2питон>
```

```
Производная в этой точке (базовый класс): Косинус: cos(1.0) = 0.5403
(venv) PS C:\Users\Nastya\OneDrive\Рабочий стол\Лаба2питон> & "c:\Users\Nastya\OneDrive\Рабочи
e/Рабочий стол/Лаба2питон/main2.py"

Выберите функцию для демонстрации:
1. Синус
2. Косинус
3. Тангенс
Ваш выбор (1-3): 2
Введите значение аргумента x (например, 1.57): п
Ошибка: введите число!
(venv) PS C:\Users\Nastya\OneDrive\Рабочий стол\Лаба2питон>
```